# Docker - Najbolje Prakse

Danijel Radaković

# 1 Docker Build

# Docker Build - Bezbednosne preporuke



- **Multi-stage build** može drastično da smanji veličinu slike i da doprinosi povećanju bezbednosti.
- Povećanje bezbednosti pomoću *multi-stage build*-a: kreirati sliku koja se koristi samo za izvršavanje aplikacije u kojoj nema *source code* fajlova kao i *build* alata. Ovim se sprecava da neko u kontjejneru izmeni source code, *build*-uje izmenjenu (maliciouznu) verziju i pokrene.
- Povodom ovog pitanja jezici koji se kompajliraju imaju prednost (Go, Rust, C++) jer se pokreću kao izvršive datoteke (*binary*) u odnosu na interpretirane jezike (JavaScript, Python) jer zahtevaju source code da bi se pokrenuli.

# Docker Build - Bezbednosne preporuke



- Zbog toga je neophodno nezavisno od jezika da se *owner* aplikacije podesi kao **root** user (`chmod`), kreirati novog korisnika u kontejneru koji će imati permisije da čita i izvršava aplikaciju (nema *write* permisiju).
- U slučaju kompajliranih jezika neophodno je izmeniti permisije izvršive datoteke, dok kod interpretiranih jezika neophodno je izmeniti sve *source code* fajlove i njihove zavisnosti.
- Preporučuje se da se koristi **slim** verzije docker slika jer pružaju manje prostora da se neka ranjivost pronađe u zavisnostima. (Primer iz *snyk* softvera, broj zavisnosti je pao sa 544 na 55).

# Docker Build - Bezbednosne preporuke



Recommendations for upgrading the base image

| | BASE IMAGE | VULNERABILITIES | SEVERITY |
|---|---|---|---|
| Current image | node:8 | 811 | 192 H 172 M 447 L |
| Major upgrades | node:10.23.2 | 544 | 56 H 55 M 433 L |
| Alternative upgrades | node:10.23.1-buster-slim | 55 | 10 H 5 M 40 L |
| | node:10.23.1-slim | 74 | 18 H 8 M 48 L |
| | node:10.23-stretch-slim | 74 | 18 H 8 M 48 L |
| | node:14.15.5-buster | 307 | 34 H 43 M 230 L |

Show less

Slika 1: Slim verzije slika imaju manje zavisnosti

# Docker Build - Bezbednosne preporuke



- Koristiti hadolint koji uočava sve propuste i nedostatke u Dockerfile-u.
- Neki od primera je *shell* ili *exec* varijante: CMD i ENTRYPOINT.
  - *shell* varijanta se ne koristi za pokretanje aplikacija jer ne prosleđuje signale podprocesima.

# Docker Build - Bezbednosne preporuke



- Korististi **clean build context**: koristiti poseban folder u kojem se nalaze svi neophodni fajlovi koji treba da budu deo *build context*-a i koristi se u *Dockerfile*-u.

```
docker buildx /files
```

- Keširanje radi bolje jer je ograničen samo na promene unutar tog foldera i igrnoriše potrebu za *Dockerignore* fajlom.
- Postoje razlličiti tipovi *build context*-a (docs).

# Dockerfile - Alias



```
FROM python
ENV update='apt-get update -qq'
ENV install='apt-get install -qq'
RUN $update && $install apt-utils \
  curl \
  gnupg
```

- Korišćenje alias komandi u *Dockerfile*-u je moguće pomoću `ENV`.
- Nije moguće koristit `alias ll=ls -alh >> .bashrc` jer je to moguće samo u interaktivnom režimu kada se nakači na kontejner pomoću `docker run -it` ili `docker exec` komandi.

# Dockerfile - Pipe



```
FROM python
SHELL ["/bin/bash", "-o", "pipefail", "-c"]
#SHELL ["/bin/ash", "-o", "pipefail", "-c"] za alpine
RUN curl -L ${SRC_URL} tar -xz
```

- Kada se koristi *pipe* u `RUN` komandi neophodno je podesiti *shell* tako da ukoliko komanda pre *pipe*-a ne završi uspešno da se ne nastavi sa izvrsavanjem *Dockerfile*-a.

# Dockerfile - ONBUILD



- Kako definisati generičke slike?
  - Na primer sliku za izvršavanje Python aplikacija u produkciju (utegnut *security*, ispoštovane najbolje prakse itd.).
  - Sve Python aplikacije u produkciji treba da koriste ovu sluku.
- Problem predstavljaju stvari koje su specifične za aplikaciju (*source code* i *dependencies*, konfiguracioni fajlovi). Kako njih ubaciti u generičku sliku?
- `ONBUILD` komanda u *Dockerfile*-u definiše trigere koji će se izvršiti prilikom nasleđivanja slike. Koristi se za definisanje generičke slike koja će se naslediti i podesiti u skladu sa potrebama. Ide dobro u kombinaciji sa *multi-stage build*-om.

# Dockerfile - ONBUILD



```
FROM python3.9-alpine3.13
LABEL maintainer "danijelradakovic@uns.ac.rs"

ONBUILD ADD requirements.txt /app
ONBUILD RUN pip install -r /app/requirements.txt
ONBUILD COPY . /app

WORKDIR /app
EXPOSE 5000
ENTRYPOINT ["python"]
CMD ["application.py"]
```

# Dockerfile - ONBUILD



- U konkretnom primeru je generička slika za Python aplikacije: fiksiran je OS, Python verzija, dok se *source code* i intaliranje *dependency*-ja radi prilikom nalseđivanja ove slike.
- Neophodno ju je samo naslediti (`FROM python-prod`) u *Dockerfile*-u konkretne aplikacije.
- Naravno, uvek se slika može naslediti i pregaziti kako bi se prilagodili potrebama aplikacije (npr. izmeniti `CMD` komandu).

# Dockerfile - Concurent Builds



```
FROM maven:3.6-jdk-8-alpine AS builder
...
FROM alpine:latest AS assets
RUN echo "Hello World" > /out/assets.html

FROM openjdk:8-jre-$alpine AS release
COPY --from=builder /app/target/app.jar
COPY --from=assets /out /assets
CMD java -jar /app/app.jar
```

- BuildKit može konkuretno da izvršava *stage*-ove. Postoji vise `--from` u nekom *stage*-u, konkuretno ce se izvrisit svaki `--from`.

# Dockerfile - Concurent Builds



- Preporuka je da `--from` bude jedni ispod drugih.
- Preporuka je da sve što treba da se kopira iz prethodno *stage*-a stavi u `/out` folder i onda kopirati `/out` folder u trenutni stage. U suprotnom bi postojale više `--from` naredbe koje bi kopirale pojedinačne fajlove ili folder koji su razbacani u prethom *stage*-u.

# Dockerfile - CI/CD



```
FROM maven:3.6-jdk-8-alpine AS builder
WORKDIR /app
COPY pom.xml
RUN mvn -e -B dependency:resolve
COPY src ./src
RUN mvn -e -B package -DskipTests

FROM openjdk:8-jre-buster AS release-buster
COPY --from=builder /app/target/app.jar
CMD java -jar /app/app.jar

FROM openjdk:8-jre-alpine AS release-alpine
COPY --from=builder /app/target/app.jar
CMD java -jar /app/app.jar
```

# Dockerfile - CI/CD (ii)



- Poželjno je imati barem poseban *stage* za svaki OS koji se koristi, npr. za debian (`release-buster`) i za alpine (`release-alpine`).
- Naredni primer je parametrizovana verzija prethodnog.

# Dockerfile - CI/CD



```dockerfile
ARG flavor=alpine

FROM maven:3.6-jdk-8-alpine AS builder
WORKDIR /app
COPY pom.xml
RUN mvn -e -B dependency:resolve
COPY src ./src
RUN mvn -e -B package -DskipTests

FROM openjdk:8-jre-${flavor} AS release
COPY --from=builder /app/target/app.jar
CMD java -jar /app/app.jar
```

# Dockerfile - CI/CD Test Stage



```dockerfile
ARG flavor=alpine

FROM maven:3.6-jdk-8-alpine AS builder
WORKDIR /app
COPY pom.xml
RUN mvn -e -B dependency:resolve
COPY src ./src
RUN mvn -e -B package -DskipTests

FROM builder AS unit-test
RUN mvn -e -B test

FROM openjdk:8-jre-${flavor} AS release
COPY --from=builder /app/target/app.jar
```

# Dockerfile - CI/CD Test Stage (ii)



```dockerfile
CMD java -jar /app/app.jar

FROM release AS integration-test
# add dependencies for integration tests
RUN apk add --no-cache curl
RUN ./test/run.sh
```

# Dockerfile - CI/CD Lint Stage



```
FROM maven:3.6-jdk-8-alpine AS builder
WORKDIR /app
COPY pom.xml
RUN mvn -e -B dependency:resolve
COPY src ./src
RUN mvn -e -B package

FROM openjdk:8-jre-alpine AS lint
RUN wget https://github.com/checkstyle/checkstyle/releases/download/
checkstyle-8.15/checkstyle-8.15-all.jar
COPY checks.xml
COPY src /src
RUN java -jar checkstyle-8.15-all.jar -c checks.xml /src
```

# Dockerfile - RUN Mount Types



```
FROM maven:3.6-jdk-8-alpine AS builder
WORKDIR /app
RUN --mount=source=pom.xml/,target=. mvn -e -B dependency:resolve
RUN --mount=source=./src mvn -e -B package -DskipTests

FROM openjdk:8-jre-$alpine AS release
COPY --from=builder /app/target/app.jar
CMD java -jar /app/app.jar
```

- `RUN` komanda ima opciju `--mount` koji kreira *bind volume* u kontejneru
- Koristi se kako bi se eliminisale *COPY* komande u *Dockerfile*-u.

# Dockerfile - RUN Mount Types



```
FROM maven:3.6-jdk-8-alpine AS builder
WORKDIR /app
RUN --mount=target=. \
    --mount=type=cache,target=/root/.m2 \
      mvn package -DoutputDirectory=/
FROM openjdk:8-jre-$alpine AS release
COPY --from=builder /app/target/app.jar
CMD java -jar /app/app.jar
```

- Još bolja opcija je da se koristi application *cache*.

# Dockerfile - RUN Mount Types



- Većina alata ima svoj *cache* folder a neki od njih su:
  - apt: `/var/lib/apt/lists`
  - go: `~/.cache/go-build`
  - go-modules: `$GOPATH/pkg/mod`
  - npm: `~/.npm`
  - pip: `~/.pip`

# Dockerfile - RUN Mount Types



- Kako rukovati sa *secret*-ima u `RUN` komandi?
  - Primer: potrebni su nam kredencijali kako bi pomoću *curl* komadne skinuli nešto sa privatnog skladišta?
  - Primer: pomoću SSH ključa treba da skinemo kod sa privatnog repozitorijuma?
- *Secret*-i se ne mogu proslediti kao argumenti slike jer se argumetni čuvaju u metapodacima slike i moguće im pristupiti preko `docker image inspect`.
- Ne mogu se kopirati pomoću `COPY` komande i zatim obrisati pomoću `RUN` komande u narednom koraku jer će `COPY` generisati novi *layer*. Za pristum pojedinačnom *layer*-u slike može se koristiti scopeo alat i videti sam sadržaj *secret*-a.

# Dockerfile - RUN Mount Types



- Pristup da se nakon RUN komande obriše secret (npr. `rm ssh-key`) će prouzrokovati da *secret* neće biti vidljiv u finalnom *layer*-u, ali će svakako biti vidlji u prethodnom *layer*-u (tj. *layer*-u `COPY` komande), kome će se opet može pristupiti pomoću skopeo alata.
- Problem nastane kad se ovakve slike okače na javni *registry* i svako može da se dočepa *secret*-a!!!
- Rešenje je korišćenje `--mount=type=secret`.

# Dockerfile - RUN Mount Types



```
FROM migration-base as execute-migration
ARG DATABASE_HOST=database
ARG MIGRATION=init
ENVENVDATABASE_HOST=${DATABASE_HOST}
MIGRATION=${MIGRATION}
RUN --mount=type=secret,id=db-psw,dst=/.secrets/db-psw,required \
    --mount=type=secret,id=db-user,dst=/.secrets/db-user,required \
    --mount=type=secret,id=db-schema,dst=/.secrets/db-schema,required \
       PATH="$PATH:/root/.dotnet/tools"; \
       export DATABASE_PASSWORD=$(cat /.secrets/db-psw); \
       export DATABASE_USERNAME=$(cat /.secrets/db-user); \
       export DATABASE_SCHEMA=$(cat /.secrets/db-schema); \
       dotnet-ef migrations add ${MIGRATION} \
         -p "${PROJECT}/${PROJECT}.csproj" \
```

# Dockerfile - RUN Mount Types (ii)



```
    --configuration Release \
    --no-build && \
  dotnet-ef database update ${MIGRATION} \
    -p "${PROJECT}/${PROJECT}.csproj" \
    --configuration Release \
    --no-build
```

# Dockerfile - Private Git Repo with SSH Forwarding



```dockerfile
FROM alpine
RUN apk add --no-cache openssh-client
RUN mkdir -p -m 0700 ~/.ssh && ssh-keyscan github.com >> ~/.ssh/
known_hosts
ARG REPO_REF=19ba7dcd9976eff829bd86541df1ba # main, develop, etc.
RUN --mount=type=ssh,required \
      git clone git@github.com:org/repo /work && cd /work \
      && git checkout -b $REPO_REF
```

```shell
# How to use it in shell
eval $(ssh-agent)
ssh-add ~/.ssh/id_rsa
docker build --ssh=default
```

# APT Package Manager



```
FROM python
RUN apt-get update -y && apt-get install -y --no-install-recommends \
      curl=7.64.0-4+deb10u2 \
      tar=1.30+dfsg-6 && \
      apt-get clean && \
      rm -rf /var/lib/apt/lists/*
```

- Kada se radi sa *package manager*-om ne treba intalirati *recomends* pakete (npr. `apt-get --no-recomends`), kao i obrisati *cache* (`/var/lib/apt/list/*`).
- Fiksrirati verzije paketa kada se instaliraju pomoću *package manager*-a.

# Maven Package Manager



```
FROM maven:3.6-jdk-8-alpine
WORKDIR /app
COPY pom.xml
RUN mvn -e -B dependency:resolve
COPY src ./src
RUN mvn -e -B package
```

- Dobra je praksa da se *dependency*-ji iz `pom.xml` fajla kesiraju tako sto ce se koristiti `dependency:resolve plugin` (usefull comands).

# Docker Build - Optimizacija



- Optimizacija kroz cache.
- Keširanje dosta jede memorije pa treba obratiti pažnju i na garbage collector.
- Spomenut je `RUN --mount=type=cache` koji koristi *bound volume* u pozaditi. *Bound volume* skladišti podatke na lokalnoj mašini što je odlično za *build* sa lokalne mašine ali problem nastaje prilikom izvršavanja u CICD jer *runner*-i koji izšavaju *job*-ove mogu biti različiti i lokalni *cache* nije isti.
- Za CI-CD se najčešće koristi remote storage backend - `inline`.

# 2 Docker Compose

# Docker Templating



- `docker config`: generiše compose fajl na osnovu compose *template*-a i i *env* fajla.

```
# env.conf.template
STAGE=prod
POSTGRES_VERSION=13
```

# Docker Templating



```yaml
# persistance.yaml
services:
  database:
    image: postgres:${POSTGRES_VERSION:-12} # default: 12
  environment:
    POSTGRES_PASSWORD_FILE: /run/secrets/database-password
    POSTGRES_USER_FILE: /run/secrets/database-username
    POSTGRES_DB_FILE: /run/secrets/database-schema
  volumes:
    - database-data:/var/lib/postgresql/data

volumes:
  database-data:
    name: clean_cadet_database_${STAGE:-dev} # default: dev
```

# Docker Templating



- `docker compose --env-file env.template.conf --file persistance.yml config > compose.yml`

```yaml
services:
  database:
    image: postgres:13
    environment:
      ...
    volumes:
      - database-data:/var/lib/postgresql/data

volumes:
  database-data:
    name: clean_cadet_database_prod
```

# 3 Docker Swarm

# Docker Swarm



- Koristi se za formiranje **swarm** klustera (umreženih servera - čvorova) i deployment servisa na kluster.
- Postoje 2 tipa čvora:
  - Master: koristi se za upravljanje klustera ( samo na *master* čvorovima mogu se izvršavati Docker komande)
    - Među *master* čvorovima postoji lider koji donosi odluke
    - Izbor novog lidera među *master* čvorovima se odvija pomću Raft protokola.
  - Slave: koriste za izvršavanje aplikacija.
- Saobraćaj se odvija kroz 2 kanala:
  - control plane: Docker komande, komunikacija koja se odvija između *master* čvorova.
  - data plane: saobraćaj samih aplikacija.

# Docker Swarm



- Preporučuje da da bude uvek neparan broj *master* čvorova zbog segmentacije mreže.
- Ne preporučuje se da ima više od 7 master čvorova jer nema benefita, samo otežava sihronizaciju.
- Infrastruktura se definiše u *yaml* fajlu koji se naziva **stack** (sličan *compose* fajlu).
  - deployment stack-a: `docker stack deploy stack.yml`
  - svaki servis mora da sadrži `deploy` sekciju u kojem se definise *deployment* samog servisa unutar klustera.
  - postoje 2 tipa *deployment*-a:
    - replicated (default): definiše broj replika i *swarm* ih raspoređuje unutar klustera,
    - global: na svakom serveru unutar klastera će se nalaziti kontejner.

# Docker Secrets



- Koristi se čuvanje osetljivih informacija i transfer do servisa kojima su te informacije potrebne (docs).

```
printf "%s" "${DATABASE_PASSWORD}" \
| docker secret create "clean_cadet_database_password_dev" - > /dev/null
printf "%s" "${DATABASE_USERNAME}" \
| docker secret create "clean_cadet_database_username_dev" - > /dev/null
printf "%s" "${DATABASE_SCHEMA}" \
| docker secret create "clean_cadet_database_schema_dev" - > /dev/null
```

- Ovi *secret*-i se mogu koristiti u bilo kom *stack*-u, ali se moraju označiti kao external (primer na sledećem slajdu).

# Docker Secrets



```yaml
services:
  database:
    image: postgres:${POSTGRES_VERSION:-13}
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/database-password
      POSTGRES_USER_FILE: /run/secrets/database-username
      POSTGRES_DB_FILE: /run/secrets/database-schema
  secrets:
    - source: database-password
      target: database-password
    - source: database-username
      target: database-username
    - source: database-schema
      target: database-schema
```

# Docker Secrets (ii)



```yaml
secrets:
  database-password:
    name: clean_cadet_database_password_dev
    external: true
  database-username:
    name: clean_cadet_database_username_dev
    external: true
  database-schema:
    name: clean_cadet_database_schema_dev
    external: true
```

# Docker Secrets



- *Secret*-i se ne mogu obrisati sve dok postoji barem jedan servis koji ih koristi!
- Za izmenu *secret*-a neophodno je:
  - obrisati sve servise koji koriste taj *secret*,
  - obrisati sam *secret*,
  - kreirati novi *secret* sa željenim sadžajem,
  - kreirati sve servise koji koriste taj *secret*.

# Docker Secrets



- Pristup koji olakšava upravljanje jeste da se u samom nazivu *secret*-a koristi verzija:
  - krerati novi *secret* sa željenim sadržajem i verzijom,
    - za svaki servis kreirati novi i obristati stari *secret*:
      - u *stack* fajlu ubaciti novu verziju i izmeniti konfiguraciju servisa da koriste novi *secret*, *swarm* će uspeti da napravi odgovarajuću izmenu (u pozadini će izvršiti sledeću komandu).
      - `docker service update --secret-rm source=<old> --secret-add source=<new>`.
  - obrisati stari secret.

# Docker Configs



- Kako rukovati sa konfiguracionim fajlovima unutar servisa koji se često menjanju. Na primer konfiguracija *nginx*-a?
  - Pomoću *docker compose*-a se ovaj problem lako rešava koristeći *bind volume*.
  - Međutim u *swarm*-u se ne koristi *bind volume* zato što se vezuje *host file system*, pa je samim tim nemoguće distibuirati konfiguracione fajlove servisima koji se izvršavaju na drugim čvorovima u klusteru.
- Jedno od rešenja je da se konfiguracioni fajlovi zapakuju u sliku kontejnera i menjati verziju slike svaki put kad je neophodno izmeniti konfiguracioni fajl (naporan posao kada se konfiguracija često menja).

# Docker Configs



- Za ovakve potrebe koristiti Docker Configs (docs) koji distriburia konfiguracione fajlove servisima kojima su neophodni.

```yaml
services:
  gateway:
    image: nginx:${VERSION-16}
    configs:
      - source: nginx.conf
        target: /etc/nginx/nginx.conf
      - source: api_gateway.conf
        target: /etc/nginx/api_gateway.conf

configs:
  nginx.conf:
```

# Docker Configs (ii)



```
    name: nginx.conf-prod
    file: ./gateway/files/config/nginx.conf # fajl mora da bude na master
nodu gde se radi deploy stack-a
  api_gateway.conf: # alternativa je da kreiraju van stack-a i imprt-uju
pomoću external
    name: api_gateway.conf-prod
    file: ./gateway/files/config/api_gateway.conf
```

# Docker Configs



```yaml
configs:
  nginx.conf:
    name: nginx.conf-prod
    file: ./gateway/files/config/nginx.conf
  api_gateway.conf:
    name: api_gateway.conf-prod
    file: ./gateway/files/config/api_gateway.conf
```

- Config će se update-ovati samo kade se promeni naziv *config*-a.
  - Primer: Ukoliko se promeni sadržaj `api_gateway.conf` fajla i uradi *deploy stack*-a neće se uraditi izmene je je naziv ostao isti (`api_gateway.conf-prod`).
  - Drugim rečima *config* je *inmutable*.
- *Config* nije moguće obrisati ako ga neki servis koristi.

# Docker Configs



- Može koristiti verzije u nazivima *config*-a kako bi se rešili ovi problemi.
- Umesto verzija moguće je koristiti *hash* sadržaja konfiguracionog fajla (kao sufiks u imenu). Na ovaj način će se naziv *config*-a menjati kada se izmeni sam sadržaj fajla.

```
configs:
  nginx.conf:
    name: nginx.conf-prod-<hash-from-nginx.conf-file>
    file: ./gateway/files/config/nginx.conf
  api_gateway.conf:
    name: api_gateway.conf-prod-<hash-from-api_gateway.conf-file>
    file: ./gateway/files/config/api_gateway.conf
```

# Docker Configs



```yaml
configs:
  nginx.conf:
    name: nginx.conf-${nginx_conf_DIGEST}
    file: ./gateway/files/config/nginx.conf
  api_gateway.conf:
    name: api_gateway.conf-${api_gateway_conf_DIGEST}
    file: ./gateway/files/config/api_gateway.conf
```

- Ideja implementacije (source):
  - Postaviti DIGEST *placeholder*-e u *stack template*-u kako bi se znalo za koje *config*-e je neophodno generisati *hash* sufiks.
  - Generiši *hash* na osnovu sadržaja fajla.
  - Uradi skraćivanje *hash* sufiksa tako da kompletan naziv *config*-a ima 64 karaktera (64 je maksimalna broj karaktera dozvoljen za naziv *config*-a).

# Docker Configs (ii)



- Izgeneriši env fajl koji mapira *placeholder*-e na *hash*.
  - `nginx_conf_DIGEST=truncedhashfromnginx`,
  - `api_gateway_conf_DIGEST=truncedhashfromapigateway`
- kombinacijom *env* fajla i *stack template*-a geniriši finalni *stack* (naredni slajd)

# Docker Configs



- Postoji već gotov kontejner koji radi generisanje env fajla (source).

```
docker create –name config-hash cleancadet/docker-config-hash:latest # kreira kontejner, ne pokreće ga
docker cp ../ config-hash # kopira sve neophodne fajlove
docker start config-hash # pokreće kontejner
docker cp config-hash:/tmp/env . # kopiraj rezultujući env fajl iz kontejnera na lokalni fajl sistem
docker rm config-hash # obriši kontejner koji više nije potreban
docker compose --env-file env \
  --file app.yml config \
  | docker stack deploy --prune -c - app-prod # kreiraj finalni stack i radi njegov deploy
rm env # obriši env fajl jer više nije potreban
```

# Docker Configs i Docker Secrets



- Ukoliko se u konfiguracionom fajlu nalaze osetljive informacije i tom slučaju treba koristi opciju `--template-driver` i napistai Go template (primer).